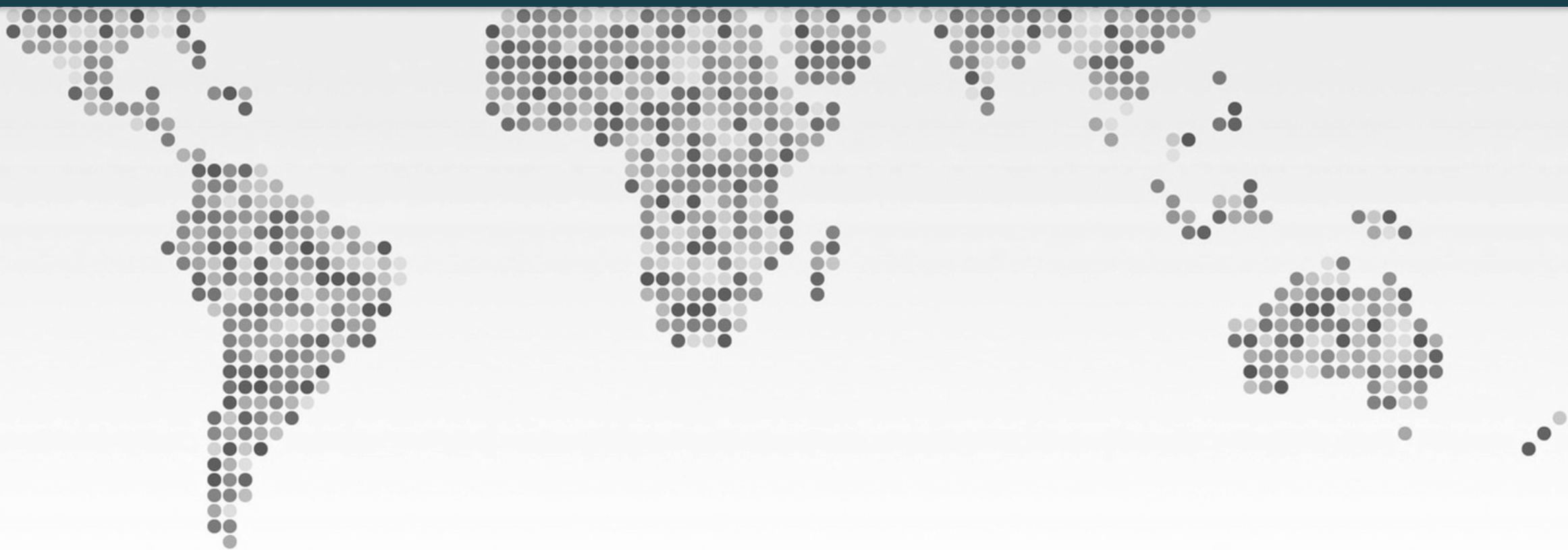


Software Engineering for Artificial Intelligence

Model Validation and Testing in the Era of Foundation Models



Software Engineering for Artificial Intelligence: LLM Evaluation

LLM Evaluation

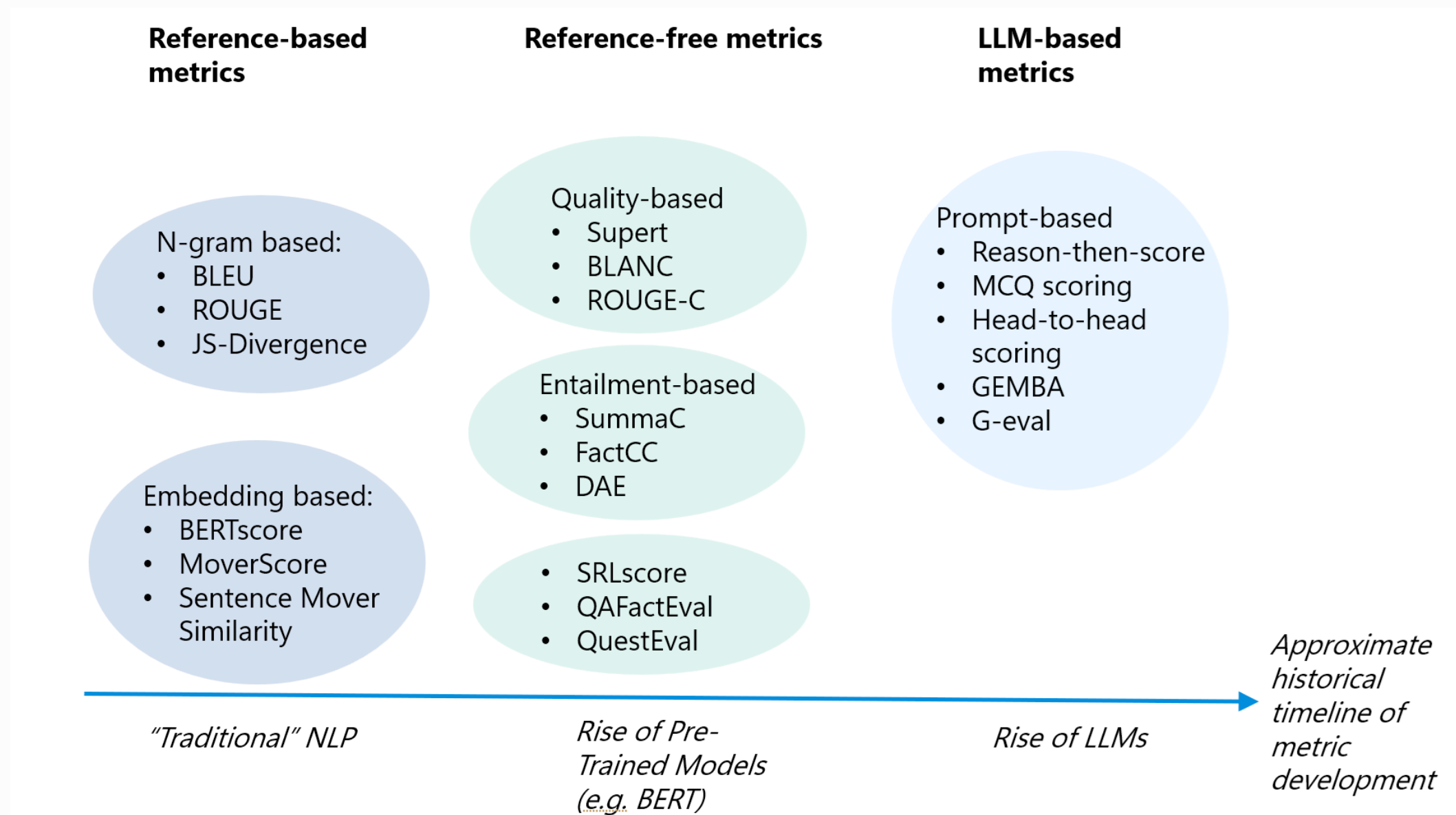
In the previous lecture, we examined validation and verification in AI-enabled systems, discussing how evaluation plays a central role in assessing system performance and reliability. We saw that, in data-driven systems, defining appropriate evaluation criteria is often more complex than in traditional software.

Large language models extend this challenge. Unlike models designed for classification or prediction with well-defined outputs, LLMs generate open-ended text across a wide range of tasks. As their capabilities expand, from summarization and translation to reasoning and dialogue, the question of how to evaluate their performance becomes increasingly central.

Evaluating LLMs therefore requires methods that can capture not only correctness, but also aspects such as coherence, relevance, factual accuracy, and overall quality. Over time, the research community has developed different approaches to approximate these dimensions, each grounded in specific assumptions about what “quality” means in generative systems.

Software Engineering for Artificial Intelligence: LLM Evaluation

LLM Evaluation



Evaluation metrics for LLM-generated content can be categorized into reference-based, reference-free, and LLM-based approaches; the timeline highlights the approximate stage in AI evolution at which each family of metrics emerged.

Software Engineering for Artificial Intelligence: LLM Evaluation

Reference-based metrics

Reference-based metrics are used to compare generated text to a reference, the human-annotated *ground truth* text. In other words, we evaluate a model by comparing what it generates with what we consider the “correct” answer. The quality of the output is estimated by measuring how closely it matches a human-written reference.

This approach is intuitive and has been widely adopted because it provides a clear and operational way to quantify performance, especially in tasks where a ground-truth answer is available.

Many of these metrics were developed for traditional NLP tasks before LLMs were developed but remain applicable to LLM-generated text.

The comparison between a generated output and its reference can be performed at different levels of granularity.

At the **token level**, metrics measure the overlap of individual words or n-grams between the generated text and the reference.

At the **phrase level**, the focus shifts to matching longer contiguous sequences or syntactic units.

At the **semantic level**, the evaluation aims to capture meaning similarity, even when different words or structures are used to express the same idea.

Reference-based metrics

BLEU Score

The BLEU (*bilingual evaluation understudy*) score evaluates the quality of machine-translated text from one natural language to another. It is typically used for machine translation tasks; however, it is also used for other tasks such as text generation, paraphrase generation, and text summarization.

The basic idea is to compute precision, the fraction of candidate words in the reference translation. Scores are calculated for individual translated segments by comparing them with a set of good-quality reference translations. Those scores are then averaged across the entire corpus to obtain an estimate of the translation's overall quality. Punctuation or grammatical correctness is not taken into account when scoring.

BLEU assumes that quality corresponds to surface overlap. This assumption works relatively well in constrained tasks like machine translation. But in open-ended text generation, where multiple semantically correct outputs may share little lexical overlap, BLEU becomes less reliable.

Reference-based metrics

ROUGE

The ROUGE (*Recall-Oriented Understudy for Gisting Evaluation*) evaluation metric measures the recall. It measures how much of the reference content is captured by the generated text.

This difference is particularly important in summarization. In machine translation, precision matters because we want the generated sentence to closely match the reference. In summarization, however, the main concern is whether the system has covered the key information from the original text. Missing important content is generally more problematic than including extra wording.

There are several variants. ROUGE-N measures n-gram overlap. ROUGE-L uses the longest common subsequence, which captures sequence-level similarity and fluency. Other variants like ROUGE-S consider skip-bigrams.

Because it emphasizes content coverage, ROUGE has become the standard automatic metric for summarization. However, like BLEU, it relies on surface overlap and does not fully capture semantic equivalence.

Reference-based metrics

JS-Divergence

The *Jensen–Shannon Divergence (JSD)* is a distribution-based metric that measures the similarity between two probability distributions. It compares the word distributions of the generated text and the reference text. Instead of counting overlapping n-grams, it evaluates the overall lexical similarity.

This difference is particularly relevant when surface overlap is not sufficient to capture content similarity. While BLEU and ROUGE focus on matching specific word sequences, JS-Divergence evaluates whether the generated text reflects a similar content distribution to the reference. Even if the wording differs, two texts may still exhibit similar probabilistic structures.

However, like other pre-neural metrics, it relies on surface-level lexical statistics. It ignores word order, syntax, and deep semantic equivalence. Two texts may have similar word distributions while conveying different meanings. Therefore, although JS-Divergence captures global content alignment better than n-gram overlap alone, it still does not fully model semantic similarity.

Semantic Similarity metrics

Semantic Similarity metrics rely on contextualized embeddings to measure the similarity between two texts. While these metrics are relatively simple, fast, and inexpensive to compute, they can exhibit poor correlation with human evaluators, lack interpretability, be inherently biased, be poorly adaptable to a wider variety of tasks, and fail to capture subtle nuances in language.

The semantic similarity between two sentences refers to how closely related their meanings are. To do that, each string is first represented as a feature vector that captures its semantics/meanings. One commonly used approach is to generate embeddings of the strings (for example, using an LLM) and then use *cosine similarity* to measure the similarity between the two embedding vectors.

Cosine similarity measures how similar two vectors are by looking at how closely they point in the same direction, regardless of their length.

{	-1	The two vectors are dissimilar
	1	The two vectors are identical

Software Engineering for Artificial Intelligence: LLM Evaluation

Semantic Similarity metrics

BERTScore

BERTSCORE: EVALUATING TEXT GENERATION WITH BERT

Tianyi Zhang^{*†‡}, Varsha Kishore[‡], Felix Wu[‡], Kilian Q. Weinberger^{†‡}, and Yoav Artzi^{†§}

[†]Department of Computer Science and [§]Cornell Tech, Cornell University
{vk352, fw245, kilian}@cornell.edu {yoav}@cs.cornell.edu

^{*}ASAPP Inc.
tzhang@asapp.com

ABSTRACT

We propose BERTSCORE, an automatic evaluation metric for text generation. Analogously to common metrics, BERTSCORE computes a similarity score for each token in the candidate sentence with each token in the reference sentence. However, instead of exact matches, we compute token similarity using contextual embeddings. We evaluate using the outputs of 363 machine translation and image captioning systems. BERTSCORE correlates better with human judgments and provides stronger model selection performance than existing metrics. Finally, we use an adversarial paraphrase detection task to show that BERTSCORE is more robust to challenging examples when compared to existing metrics.

1 INTRODUCTION

Automatic evaluation of natural language generation, for example in machine translation and caption generation, requires comparing candidate sentences to annotated references. The goal is to evaluate semantic equivalence. However, commonly used methods rely on surface-form similarity only. For example, BLEU (Papineni et al., 2002), the most common machine translation metric, simply counts n -gram overlap between the candidate and the reference. While this provides a simple and general measure, it fails to account for meaning-preserving lexical and compositional diversity.

In this paper, we introduce BERTSCORE, a language generation evaluation metric based on pre-trained BERT contextual embeddings (Devlin et al., 2019). BERTSCORE computes the similarity of two sentences as a sum of cosine similarities between their tokens' embeddings.

BERTSCORE addresses two common pitfalls in n -gram-based metrics (Banerjee & Lavie, 2005). First, such methods often fail to robustly match paraphrases. For example, given the reference *people like foreign cars*, BLEU and METEOR (Banerjee & Lavie, 2005) incorrectly give a higher score to *people like visiting places abroad* compared to *consumers prefer imported cars*. This leads to performance underestimation when semantically-correct phrases are penalized because they differ from the surface form of the reference. In contrast to string matching (e.g., in BLEU) or matching heuristics (e.g., in METEOR), we compute similarity using contextualized token embeddings, which have been shown to be effective for paraphrase detection (Devlin et al., 2019). Second, n -gram models fail to capture distant dependencies and penalize semantically-critical ordering changes (Isozaki et al., 2010). For example, given a small window of size two, BLEU will only mildly penalize swapping of cause and effect clauses (e.g. *A because B* instead of *B because A*), especially when the arguments A and B are long phrases. In contrast, contextualized embeddings are trained to effectively capture distant dependencies and ordering.

We experiment with BERTSCORE on machine translation and image captioning tasks using the outputs of 363 systems by correlating BERTSCORE and related metrics to available human judgments. Our experiments demonstrate that BERTSCORE correlates highly with human evaluations. In machine translation, BERTSCORE shows stronger system-level and segment-level correlations with human judgments than existing metrics on multiple common benchmarks and demonstrates

MoverScore

MoverScore: Text Generation Evaluating with Contextualized Embeddings and Earth Mover Distance

Wei Zhao[†], Maxime Peyrard[†], Fei Liu[†], Yang Gao[†], Christian M. Meyer[†], Steffen Eger[†]

[†]Computer Science Department, Technische Universität Darmstadt, Germany

[‡]Computer Science Department, University of Central Florida, US

zhao@aiphes.tu-darmstadt.de, maxime.peyrard@epfl.ch

feiliu@cs.ucf.edu, yang.gao@rhul.ac.uk

meyer@ukp.informatik.tu-darmstadt.de

eger@aiphes.tu-darmstadt.de

Abstract

A robust evaluation metric has a profound impact on the development of text generation systems. A desirable metric compares system output against references based on their semantics rather than surface forms. In this paper we investigate strategies to encode system and reference texts to devise a metric that shows a high correlation with human judgment of text quality. We validate our new metric, namely MoverScore, on a number of text generation tasks including summarization, machine translation, image captioning, and data-to-text generation, where the outputs are produced by a variety of neural and non-neural systems. Our findings suggest that metrics combining contextualized representations with a distance measure perform the best. Such metrics also demonstrate strong generalization capability across tasks. For ease-of-use we make our metrics available as web service.¹

1 Introduction

The choice of evaluation metric has a significant impact on the assessed quality of natural language outputs generated by a system. A desirable metric assigns a single, real-valued score to the system output by comparing it with one or more reference texts for content matching. Many natural language generation (NLG) tasks can benefit from robust and unbiased evaluation, including text-to-text (*machine translation* and *summarization*), data-to-text (*response generation*), and image-to-text (*captioning*) (Gatt and Krahmer, 2018). Without proper evaluation, it can be difficult to judge on system competitiveness, hindering the development of advanced algorithms for text generation.

It is an increasingly pressing priority to develop better evaluation metrics given the recent advances in neural text generation. Neural models provide

the flexibility to copy content from source text as well as generating unseen words (See et al., 2017). This aspect is hardly covered by existing metrics. With greater flexibility comes increased demand for unbiased evaluation. Diversity-promoting objectives make it possible to generate diverse natural language descriptions (Li et al., 2016; Wiseman et al., 2018). But standard evaluation metrics including BLEU (Papineni et al., 2002) and ROUGE (Lin, 2004) compute the scores based primarily on n -gram co-occurrence statistics, which are originally proposed for diagnostic evaluation of systems but not capable of evaluating text quality (Reiter, 2018), as they are not designed to measure if, and to what extent, the system and reference texts with distinct surface forms have conveyed the same meaning. Recent effort on the applicability of these metrics reveals that while compelling text generation system ascend on standard metrics, the text quality of system output is still hard to be improved (Böhm et al., 2019).

Our goal in this paper is to devise an automated evaluation metric assigning a single holistic score to any system-generated text by comparing it against human references for content matching. We posit that it is crucial to provide a holistic measure attaining high correlation with human judgments so that various neural and non-neural text generation systems can be compared directly. Intuitively, the metric assigns a perfect score to the system text if it conveys the same meaning as the reference text. Any deviation from the reference content can then lead to a reduced score, e.g., the system text contains more (or less) content than the reference, or the system produces ill-formed text that fails to deliver the intended meaning.

We investigate the effectiveness of a spectrum of distributional semantic representations to encode system and reference texts, allowing them to be compared for semantic similarity across

¹Our code is publicly available at <http://tiny.cc/vsqtbz>

SMS

Sentence Mover's Similarity: Automatic Evaluation for Multi-Sentence Texts

Elizabeth Clark^{1*}, Asli Celikyilmaz², Noah A. Smith^{1,3}

¹Paul G. Allen School of Computer Science & Engineering, University of Washington

²Microsoft Research

³Allen Institute for Artificial Intelligence

{eacclark7, nasmith}@cs.washington.edu aslicel@microsoft.com

Abstract

For evaluating machine-generated texts, automatic methods hold the promise of avoiding collection of human judgments, which can be expensive and time-consuming. The most common automatic metrics, like BLEU and ROUGE, depend on exact word matching, an inflexible approach for measuring semantic similarity. We introduce methods based on *sentence mover's similarity*; our automatic metrics evaluate text in a continuous space using word and sentence embeddings. We find that sentence-based metrics correlate with human judgments significantly better than ROUGE, both on machine-generated summaries (average length of 3.4 sentences) and human-authored essays (average length of 7.5). We also show that sentence mover's similarity can be used as a reward when learning a generation model via reinforcement learning; we present both automatic and human evaluations of summaries learned in this way, finding that our approach outperforms ROUGE.

1 Introduction

Automatic text evaluation reduces the need for human evaluations, which can be expensive and time-consuming to collect, particularly when evaluating long, multi-sentence texts. Automatic metrics allow faster measures of progress when training and testing models and easier development of text generation systems.

However, existing automatic metrics for evaluating text are problematic. Due to their computational efficiency, metrics based on word-matching are common, such as ROUGE (Lin, 2004) for summarization, BLEU (Papineni et al., 2002) for machine translation, and METEOR (Banerjee and Lavie, 2005) or CIDEr (Vedantam et al., 2015) for image captioning. Nevertheless, these metrics of-

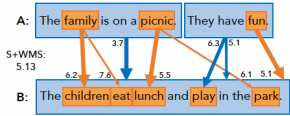


Figure 1: An illustration of S+WMS (a sentence mover's similarity metric that uses both word and sentence embeddings) between two documents. This metric finds the minimal cost of “moving” both the word embeddings (orange) and the sentence embeddings (blue) in Document A to those in Document B. An arrow's width is the proportion of the embedding's weight being moved, and its label is the Euclidean distance. Here we show only the highest weighted connections.

ten fail to capture information that has been reworded or reordered from the reference text, as shown in Kilickaya et al. (2017) and Table 1.¹ They have also been found to correlate weakly with human judgments (Liu et al., 2016; Novikova et al., 2017).

To avoid these shortcomings, word mover's distance (WMD; Kusner et al., 2015) can be used to evaluate text in a continuous space using pre-trained word embeddings instead of relying on exact word matching. WMD has been used successfully for tasks including image caption evaluation (Kilickaya et al., 2017), automatic essay evaluation (Tashu and Horváth, 2018), and affect detection (Alshahrani et al., 2017). This bag-of-embeddings approach is flexible but fails to reflect the grouping of words and ideas, a shortcoming that becomes more problematic as the length of the document grows.

We modify WMD for evaluating multi-sentence texts by basing the score on sentence embeddings (§3), giving it access to higher-level representa-

¹For readability, we scale ROUGE scores by a factor of 100 and sentence mover's metrics by a factor of 1000.

^{*}Work done while author was at Microsoft Research.

Software Engineering for Artificial Intelligence: LLM Evaluation

Reference-based metrics

BLEU Score

ROUGE

JS-Divergence

BERTScore

MoverScore

SMS

While effective in constrained settings, reference-based metrics exhibit clear limitations when applied to modern generative systems. By measuring similarity to a single reference, they tend to penalize valid paraphrases and alternative formulations that differ lexically but preserve meaning. In tasks where multiple outputs can be equally correct, similarity becomes an imperfect proxy for quality.

Moreover, the reliability of these metrics depends heavily on the quality and completeness of the reference itself. If the reference is narrow, incomplete, or stylistically biased, the metric inherits those limitations. As generative tasks become more open-ended, reference-based evaluation becomes increasingly fragile.

Most importantly, similarity does not necessarily imply usefulness, correctness, or reasoning quality. A response can closely match a reference while still being shallow or incomplete.

This raises a fundamental question: *what happens when no reliable reference exists?*

Software Engineering for Artificial Intelligence: LLM Evaluation

Reference-free metrics

Reference-free (context-based) metrics produce a score for the generated text and do not rely on ground truth. Evaluation is based on the context or source document. Many of these metrics were developed to address the challenge of creating ground truth data.

These methods tend to be newer than reference-based techniques, reflecting the growing demand for scalable text evaluation as Pre-trained Models became powerful.

Quality-based metrics

Quality-based metrics assess whether a generated summary captures the most relevant and informative content from the source document, without relying on a gold reference. Instead of comparing against ground truth, they evaluate how well the summary reflects salient information in the input text.

SUPERT builds a pseudo-reference automatically by identifying the most salient sentences in the source document using contextual embeddings. The generated summary is then compared to this pseudo-reference to measure semantic similarity and content coverage.

BLANC evaluates how effectively a summary reconstructs masked tokens in the original document. If the summary significantly improves reconstruction accuracy, it is considered informative and useful.

ROUGE-C modifies ROUGE by comparing the summary directly to the source document rather than to a human reference. It measures how much content from the source is covered by the summary.

Reference-free metrics

Entailment-Based metrics

Entailment-based metrics evaluate whether the information in a generated summary is logically supported by the source document. The source text is treated as the *premise* and the generated summary (or its individual statements) as the *hypothesis*.

The core idea is to determine whether the hypothesis is entailed by the premise, contradicts it, or is unsupported by it. In summarization evaluation, this framework is used to detect factual inconsistencies and hallucinations.

SummaC measures the degree of entailment between the source document and the generated summary. It aggregates sentence-level entailment predictions to produce a document-level consistency score, providing a fine-grained assessment of factual alignment.

FactCC is a supervised entailment-based model trained specifically to detect factual inconsistencies in summaries. It generates synthetic factual errors during training and learns to classify whether a summary sentence is supported by the source document.

DAE (Dependency Arc Entailment) evaluates factual consistency at a finer structural level by checking whether dependency relations (e.g., subject–verb–object arcs) in the summary are entailed by the source. This allows it to detect more subtle factual errors involving entity roles and relations.

Reference-free metrics

Factuality-, QA-, and QG-Based Metrics

These metrics focus on verifying whether the generated text contains factually correct information grounded in the source document. Rather than measuring overlap or general consistency, they aim to detect hallucinations and unsupported claims at a finer semantic level.

They typically operate by extracting structured information, generating questions, or verifying the consistency of answers between the source and the generated text. The central goal is to assess factual correctness and content preservation.

SRLScore evaluates factual consistency by extracting predicate–argument structures (who did what to whom) from both the source and the generated text using semantic role labeling. It then compares these structures to determine whether the summary preserves the core events and relations expressed in the source.

QAFactEval generates questions based on the source document and answers them using the generated summary. If the answers derived from the summary match those from the source, the summary is considered factually consistent.

QuestEval generates questions from both the source and the summary and checks whether each text can answer the other's questions. This bidirectional QA framework measures both relevance and factual consistency, ensuring that key information is preserved.

Software Engineering for Artificial Intelligence: LLM Evaluation

Reference-free metrics

Quality-based metrics

Factuality-, QA-, and QG-Based Metrics

Entailment-Based metrics

Although reference-free metrics appear to overcome the constraints of gold references, they introduce their own limitations. Many of these methods rely on auxiliary models such as QA systems, making their reliability dependent on the performance and biases of those underlying models. Errors can propagate from the evaluation model itself.

Furthermore, these metrics often reduce evaluation to binary or narrow judgments, such as “consistent” versus “inconsistent.” This simplification fails to capture the multidimensional nature of quality in generative systems. A response may be factually grounded yet unhelpful, incomplete, or poorly reasoned.

As tasks grow more complex and generative models become more capable, it becomes clear that removing the reference does not eliminate the deeper challenge. Even without a gold standard, evaluation still requires deciding what counts as a correct or high-quality answer.

The Oracle Problem

The limitations of both reference-based and reference-free metrics point to a deeper structural challenge. The issue is not merely whether we evaluate against a gold reference or against the source context. The more fundamental problem is that generative systems do not operate in a space with a single correct answer.

In traditional software systems, correctness can be verified against a predefined oracle: for a given input, there is an expected output. In contrast, generative models produce responses that may vary in wording, structure, depth, and reasoning, while remaining valid. Multiple outputs can satisfy the same task in different yet acceptable ways.

This directly challenges reference-based evaluation. When quality is reduced to similarity with a single reference, semantically valid alternatives may be penalized simply because they diverge lexically or structurally from the gold example. In open-ended tasks, the reference is not a ground truth but only one possible realization among many.

Moreover, quality in generative AI is inherently multidimensional. It may involve factual accuracy, coherence, completeness, reasoning quality, stylistic appropriateness, usefulness, and alignment with user intent. These dimensions do not neatly collapse into a single binary distinction between correct and incorrect.

As a result, both reference-based and reference-free approaches struggle because they attempt to approximate quality through simplified signals—similarity, consistency, or factual support—while the underlying concept of correctness remains fluid and context-dependent.

Non-Determinism in LLMs

This challenge is further compounded by the non-deterministic nature of large language models. Given the same input prompt, an LLM may generate multiple different outputs depending on decoding strategies, temperature settings, sampling procedures, or even minor model updates.

Unlike deterministic systems, LLMs define a probability distribution over possible continuations. Each generation is a sample from that distribution. As a result, the model does not produce a single canonical answer but a range of plausible responses, each with varying degrees of quality.

The variability is not necessarily a flaw; it is a feature of probabilistic generation that enables diversity and creativity. However, it complicates evaluation. If the same prompt can yield multiple valid responses, then evaluating a single output provides only a partial and potentially noisy estimate of the model's performance.

Output variance introduces instability into verification. Performance becomes distributional rather than point-based, and quality must be assessed across possible generations rather than through deterministic matching.

In this setting, evaluation can no longer rely on fixed expected outputs or strict verification. It must instead accommodate variability, uncertainty, and the inherently probabilistic nature of generative models.

Evaluation Under Uncertainty

We have seen that both reference-based and reference-free metrics, despite their differences, share a common limitation: they reduce quality to a single measurable signal, while the actual quality of a generative response is multidimensional and context-dependent.

At the same time, the non-deterministic nature of LLMs means that a single evaluation of a single output is often not enough. The same prompt can yield different responses across runs, making point-based evaluation unreliable and unsuitable.

These two challenges make it clear that traditional evaluation frameworks are insufficient for modern generative systems. Metrics designed to measure similarity to a fixed reference, or consistency with a source document, were built on the assumption that quality is a verifiable property. For LLMs operating in open-ended settings, this assumption no longer holds.

Evaluation must therefore shift from verification to judgment: from checking whether an output matches an expected form, to assessing whether it is useful, accurate, coherent, and appropriate in context. This shift requires approaches that can handle semantic variability, accommodate multiple valid outputs, and scale without relying on expensive human annotation for every sample.

Software Engineering for Artificial Intelligence: LLM Evaluation

Three Approaches to LLM Evaluation

Addressing the oracle problem and output variability requires moving beyond any single metric or technique. Three complementary approaches have emerged in practice, each targeting a different aspect of the evaluation challenge.

Benchmarking

Standardized tasks and scoring criteria to measure model capabilities at scale, across many inputs and in a reproducible way.

LLM-as-Judge

Using large language models as flexible evaluators, capable of assessing semantic quality without a gold reference.

Constrain & Guardrails

Reducing output variability at the source through prompt engineering and structured output controls.

These approaches are not mutually exclusive. In practice, robust evaluation combines all three depending on the task, the available resources, and the quality dimensions that matter.

Software Engineering for Artificial Intelligence: LLM Evaluation

Benchmarking

Whats?

Why?

Types of Benchmarks

A benchmark is a standardized collection of tasks and inputs—such as multiple choice questions, reasoning problems, or code snippets—designed to measure the capabilities of a model in a reproducible and comparable way.

From a software engineering perspective, a benchmark can be thought of as a test suite for a model's general or domain-specific behavior. However, as with any test suite, passing all tests does not guarantee correctness in production: a model can score well on a benchmark while failing on inputs that fall outside its coverage.

One important property of benchmarks is that they track the evolution of the state of the art through leaderboards. As models improve, a well-known phenomenon emerges: **saturation**, when top models approach or exceed human-level performance on a benchmark, which signals that the benchmark has lost its discriminative power and new, harder benchmarks are needed.

Benchmarking

Whats?

Why?

Types of Benchmarks

Benchmarks serve different purposes depending on who is using them. It is useful to distinguish between two main perspectives.

For **model builders**, benchmarks are a tool for controlled experimentation. They allow developers to run ablations, testing how specific design choices, such as architecture changes or different training data, affect performance relative to a known baseline. During training, benchmarks are also used to monitor learning progress and detect regressions or unexpected performance drops early.

For **model users**, benchmarks are a selection tool. When choosing a model for a specific use case, such as code generation, mathematical reasoning, or document summarization, benchmark rankings provide a structured basis for comparison that goes beyond informal testing.

Benchmarking

Whats?

Why?

Types of Benchmarks

Benchmark scores are aggregated over many standardized inputs, which makes them inherently more robust than evaluating a single output: a model that performs well consistently across hundreds of tasks is a more reliable signal than one that produces a good response to a specific prompt.

This also directly addresses the two core challenges we identified: non-determinism is mitigated by shifting evaluation from individual outputs to aggregate behavior, and the oracle problem is sidestepped by focusing on tasks where correctness is defined and automatically verifiable.

Software Engineering for Artificial Intelligence: LLM Evaluation

Benchmarking

Whats?

Why?

Types of Benchmarks

Benchmarks cover a wide range of capabilities. A first important category groups tasks that test **knowledge and reasoning**.

MMLU-Pro

GPQA

HellaSwag

These benchmarks evaluate what a model knows and how well it can reason about it. They include general knowledge tests across academic subjects, common sense reasoning, and scientific problem solving at various levels of difficulty.

A second category targets **STEM and coding** capabilities

HumanEval

EvalPlus

SweBench

In these benchmarks, the correctness is defined and automatically verifiable through execution or formal checking. They are particularly well suited to address the oracle problem, since a correct answer is unambiguous.

Benchmarking

Whats?

Why?

Types of Benchmarks

As LLMs are increasingly deployed in complex, multi-step workflows, a new generation of benchmarks has emerged to evaluate **functional and agentic capabilities**: how well a model follows instructions, uses tools, and completes tasks that require planning and interaction.

Beyond individual task completion, some benchmarks evaluate **dynamic and adaptive behavior**: how a model performs in environments that require long-horizon planning, adaptation, and reasoning under uncertainty.

This evolution in benchmark design reflects a broader shift in what we expect from LLMs, from answering questions in isolation to acting as agents in complex environments.

Benchmarking - Metrics

Benchmarks like HumanEval evaluate code generation by running the model's output against a set of unit tests. This introduces a challenge: due to non-determinism, a model may produce a correct solution on some runs and an incorrect one on others. Evaluating a single output is therefore an unreliable signal.

However, non-determinism remains a challenge. Since LLM inference is an autoregressive sampling process, the same prompt can yield different outputs across runs. Evaluating a single generated solution provides an unstable and potentially misleading signal about the model's actual capability.

pass@k addresses this by shifting from point evaluation to distributional evaluation. The model generates n total candidate solutions for a given problem. The metric estimates the probability that at least k of those candidates are correct, using the following unbiased estimator:

$$\text{pass@k} = 1 - \frac{\binom{n-c}{k}}{\binom{n}{k}}$$

n = total samples generated

c = number of correct samples

k = candidates considered

Software Engineering for Artificial Intelligence: LLM Evaluation

Benchmarking - Metrics

Benchmarks like HumanEval evaluate code generation by running the model's output against a set of unit tests. This introduces a challenge: due to non-determinism, a model may produce a correct solution on some runs and an incorrect one on others. Evaluating a single output is therefore an unreliable signal.

However, non-determinism remains a challenge. Since LLM inference is an autoregressive sampling process, the same prompt can yield different outputs across runs. Evaluating a single generated solution provides an unstable and potentially misleading signal about the model's actual capability.

pass@k addresses this by shifting from point evaluation to distributional evaluation. The model generates n total candidate solutions for a given problem. The metric estimates the probability that at least k of those candidates are correct, using the following unbiased estimator:

Example. Consider a model generating solutions to a problem where, out of $n=10$ samples, $c=3$ pass all unit tests. With $k=1$, $\text{pass}@1 = 0.30$: the probability that a single randomly drawn sample is correct. With $k=5$, applying the unbiased estimator yields $\text{pass}@5 \approx 0.92$: by considering 5 candidates, the probability of at least one correct solution rises substantially. This distributional view provides a more robust and statistically grounded estimate of model capability than any single-output evaluation.

Benchmarking Pitfalls

The most significant risk is **data contamination**: if benchmark questions end up in the training data, even unintentionally, the model learns to recognize them rather than genuinely solving them. In this scenario, a high score reflects memorization rather than capability (much like a student who has seen the exam questions in advance).

A related issue is **prompt sensitivity and overfitting**: benchmark scores can vary significantly with minimal changes in phrasing or formatting. A model optimized for a specific benchmark format may score well without possessing the underlying skill the benchmark is meant to measure.

Benchmarks are also **static snapshots**, and as models improve they tend to saturate, reaching ceiling scores where top models become indistinguishable. At that point, the benchmark loses its discriminative power and needs to be replaced with harder, more demanding tasks.

Benchmarks are a necessary condition for monitoring progress, but they are not enough...

Software Engineering for Artificial Intelligence: LLM Evaluation

LLM-as-Judge

Benchmarks are a necessary condition for monitoring progress, but they are not sufficient. They are the preferred evaluation tool whenever correctness can be defined through rules or automated tests, as in code generation, mathematics, or structured reasoning.

However, many real-world tasks such as summarization, creative writing, or complex question answering require alignment with human preferences, a dimension that no automatic metric can fully capture. It is precisely for these cases that **LLM-as-Judge** emerges as a necessary complementary tool.

The idea is to use a large language model as an automated evaluator: given a prompt and a generated response, the judge model produces a quality assessment according to explicit criteria. Rather than measuring surface overlap or executing tests, the judge reasons about the response much like a human reviewer would.

From a software engineering perspective, LLM-as-Judge acts as an automated oracle for tasks where no formal specification exists. It addresses the limitations of n-gram metrics such as BLEU and ROUGE, which measure textual overlap but not semantic quality, and provides a scalable alternative to human annotation, which is costly, slow, and often inconsistent on tasks requiring complex reasoning.

LLM-as-Judge - How to Implement an LLM Judge

LLM judges can be implemented in different ways depending on the available resources and the required reliability of the evaluation signal.

Prompted judges

Instruct a frontier model to act as an evaluator through a designed prompt, without any additional training. This is the most common approach in practice due to its flexibility and low setup cost.

Fine-tuned judges

These are models trained specifically on human preference datasets, aligning their judgments to expert annotations for a given domain. They tend to be more consistent than prompted judges but require labeled data and are less generalizable.

Multi-agent judges

These use an ensemble of LLM agents that independently evaluate the same output and then reconcile their assessments. This approach increases robustness by reducing the influence of any single model's biases.

LLM-as-Judge - How to Implement an LLM Judge

LLM judges can be implemented in different ways depending on the available resources and the required reliability of the evaluation signal.

Prompted judges

Fine-tuned judges

Multi-agent judges

Regardless of implementation, judges can operate in two scoring modes.

Pairwise scoring presents the judge with two candidate responses and asks which is better, this tends to correlate more strongly with human preferences and avoids the difficulty of assigning absolute scores.

Rating scale scoring asks the judge to assign a numeric score based on explicit criteria such as correctness, fluency, or conciseness, which is more practical when comparing many outputs at scale.

LLM-as-Judge - Prompting Strategies

The reliability of an LLM judge depends on how it is prompted. Two strategies are particularly effective in practice.

Chain-of-thought

Instructs the judge to produce a structured reasoning trace before emitting a final verdict. Rather than assigning a score directly, the model is asked to analyze the response step by step (identifying strengths, weaknesses, and specific errors) before committing to a judgment. This reduces superficial scoring and makes the evaluation more interpretable and auditable.

Reference-based

Provides the judge with a correct reference answer alongside the generated response. This is particularly useful when factual correctness matters: the judge is asked to assess not just whether the response is coherent, but whether it is accurate relative to a known ground truth. This combines the structure of reference-based evaluation with the flexibility of LLM judgment.

Biases in LLM Judges

LLM judges are powerful but not neutral. Research has documented systematic biases that can undermine the reliability of evaluations if left unaddressed.

Positional bias

Refers to the tendency to favor whichever response appears first in a pairwise comparison, regardless of its actual quality. This means that a single pairwise evaluation may not be reliable; the outcome can change simply by swapping the order of the two responses.

Verbosity bias

It causes judges to reward longer, more elaborate responses even when conciseness would be more appropriate. A model that produces verbose output may consistently receive higher scores independently of its actual quality.

Self-enhancement bias

It describes a model's tendency to favor outputs that resemble its own style or that of architecturally similar models, thereby introducing a systematic advantage for certain model families.

Biases in LLM Judges

LLM judges are powerful but not neutral. Research has documented systematic biases that can undermine the reliability of evaluations if left unaddressed.

Positional bias

Verbosity bias

Self-enhancement bias

Beyond these well-known biases, LLM judges also exhibit internal inconsistency: the same judge may produce different scores for the same input across multiple runs, making the evaluation signal inherently noisy. This instability is a direct consequence of the non-determinism we discussed earlier, and it applies to the judges just as much as to the model being evaluated.

If LLM judges are themselves subject to bias and inconsistency, a natural question arises: how do we know whether a judge is reliable? **JudgeBench** addresses this directly by treating the judge as the system under evaluation.

JudgeBench is a benchmark designed to assess the critical reasoning capability of LLM judges. It contains 350 challenging response pairs in STEM domains (mathematics, coding, and science) where errors are subtle and difficult to detect even for human experts. This makes it possible to measure whether a judge can identify genuinely incorrect reasoning, not just obvious mistakes.

Software Engineering for Artificial Intelligence: LLM Evaluation

Reward Models

Alongside generative LLM judges, a distinct class of evaluators exists: **Reward Models**. Originally developed for alignment through Reinforcement Learning from Human Feedback (RLHF), they can also serve as evaluation oracles.

A reward model is a discriminative model trained on human preference data to assign a scalar quality score to a prompt-response pair. Unlike a generative judge, it does not produce an explanation; it performs a single forward pass and returns a numeric score, making it deterministic, fast, and reproducible.

This design has practical advantages over LLM-as-Judge: reward models are significantly faster, less sensitive to prompt formatting, and free from positional or verbosity biases. Empirical results show that specialized reward models can outperform much larger generative judges, making it feasible to use lighter models to evaluate stronger ones.

Reward models are fundamental in RLHF training pipelines, but are less commonly used as general-purpose evaluators. Building one requires expensive domain-specific fine-tuning, while an LLM judge can be deployed immediately through prompt engineering. They also generalize poorly outside the distribution of their preference training data, making them less flexible than generative judges in dynamic evaluation scenarios.

Software Engineering for Artificial Intelligence: LLM Evaluation

Constrain & Guardrails

So far we have discussed how to evaluate outputs after they are generated. A different and often more practical approach is to act earlier in the process: by constraining what the model can generate, we transform an ambiguous task into one where correctness can be checked automatically.

The underlying idea is engineering-driven. In the absence of a formal oracle, constraints serve as an artificial specification, that is, a set of rules that the output must satisfy, against which testing becomes straightforward and “deterministic”.

In practice, this can be achieved at different levels.

Prompt-based

At the prompt level, explicit instructions are used to force the model toward parsable outputs, such as requiring only numeric responses or prohibiting abbreviations.

Few-shot prompting

Few-shot examples implicitly define the expected output schema without listing rules explicitly

Structured generation

At the generation level, libraries like Outlines use grammars and finite-state automata to enforce syntactic constraints directly during decoding, making non-compliant outputs structurally impossible.

Software Engineering for Artificial Intelligence: LLM Evaluation

Constrain & Guardrails

So far we have discussed how to evaluate outputs after they are generated. A different and often more practical approach is to act earlier in the process: by constraining what the model can generate, we transform an ambiguous task into one where correctness can be checked automatically.

The underlying idea is engineering-driven. In the absence of a formal oracle, constraints serve as an artificial specification, that is, a set of rules that the output must satisfy, against which testing becomes straightforward and “deterministic”.

In practice, this can be achieved at different levels.

Prompt-based

Few-shot prompting

Structured generation

Constraints and guardrails are not a testing strategy in themselves. Rather, they are engineering tools that make testing easier and more reliable: by reducing output variability and enforcing verifiable formats, they create the conditions under which automatic evaluation becomes more tractable.

Today we are also looking at other aspects (which you will see in the next lectures...)

Software Engineering for Artificial Intelligence: Verification and Validation

Before leaving... What Should You Remember?

Software Engineering for Artificial Intelligence: LLM Evaluation

LLM Evaluation

Reference-based metrics

Reference-free metrics

LLM-based metrics

N-gram based:

- BLEU
- ROUGE
- JS-Divergence

Embedding based:

- BERTScore
- MoverScore
- Sentence Mover Similarity

Quality-based

- Supert
- BLANC
- ROUGE-C

Entailment-based

- SummaC
- FactCC
- DAE

SRLscore

- QAFactEval
- QuestEval

Prompt-based

- Reason-then-score
- MCQ scoring
- Head-to-head scoring
- GEMBA
- G-eval

"Traditional" NLP

Rise of Pre-Trained Models (e.g. BERT)

Rise of LLMs

Approximate historical timeline of metric development

Evaluation metrics for LLM-generated content can be categorized into reference-based, reference-free, and LLM-based approaches; the timeline highlights the approximate stage in AI evolution at which each family of metrics emerged.

Software Engineering for Artificial Intelligence: LLM Evaluation

Three Approaches to LLM Evaluation

Addressing the oracle problem and output variability requires moving beyond any single metric or technique. Three complementary approaches have emerged in practice, each targeting a different aspect of the evaluation challenge.

Benchmarking

Standardized tasks and scoring criteria to measure model capabilities at scale, across many inputs and in a reproducible way.

LLM-as-Judge

Using large language models as flexible evaluators, capable of assessing semantic quality without a gold reference.

Constrain & Guardrails

Reducing output variability at the source through prompt engineering and structured output controls.

These approaches are not mutually exclusive. In practice, robust evaluation combines all three depending on the task, the available resources, and the quality dimensions that matter.

Software Engineering for Artificial Intelligence: LLM Evaluation

LLM-as-Judge - How to Implement an LLM Judge

LLM judges can be implemented in different ways depending on the available resources and the required reliability of the evaluation signal.

Prompted judges

Fine-tuned judges

Multi-agent judges

Regardless of implementation, judges can operate in two scoring modes.

Pairwise scoring presents the judge with two candidate responses and asks which is better, this tends to correlate more strongly with human preferences and avoids the difficulty of assigning absolute scores.

Rating scale scoring asks the judge to assign a numeric score based on explicit criteria such as correctness, fluency, or conciseness, which is more practical when comparing many outputs at scale.

Software Engineering for Artificial Intelligence: LLM Evaluation

Reference-based metrics

BLEU Score

ROUGE

JS-Divergence

BERTScore

MoverScore

SMS

While effective in constrained settings, reference-based metrics exhibit clear limitations when applied to modern generative systems. By measuring similarity to a single reference, they tend to penalize valid paraphrases and alternative formulations that differ lexically but preserve meaning. In tasks where multiple outputs can be equally correct, similarity becomes an imperfect proxy for quality.

Moreover, the reliability of these metrics depends heavily on the quality and completeness of the reference itself. If the reference is narrow, incomplete, or stylistically biased, the metric inherits those limitations. As generative tasks become more open-ended, reference-based evaluation becomes increasingly fragile.

Most importantly, similarity does not necessarily imply usefulness, correctness, or reasoning quality. A response can closely match a reference while still being shallow or incomplete.

This raises a fundamental question: *what happens when no reliable reference exists?*

Software Engineering for Artificial Intelligence: LLM Evaluation

Benchmarking - Metrics

Benchmarks like HumanEval evaluate code generation by running the model's output against a set of unit tests. This introduces a challenge: due to non-determinism, a model may produce a correct solution on some runs and an incorrect one on others. Evaluating a single output is therefore an unreliable signal.

However, non-determinism remains a challenge. Since LLM inference is an autoregressive sampling process, the same prompt can yield different outputs across runs. Evaluating a single generated solution provides an unstable and potentially misleading signal about the model's actual capability.

pass@k addresses this by shifting from point evaluation to distributional evaluation. The model generates *n* total candidate solutions for a given problem. The metric estimates the probability that at least *k* of those candidates are correct, using the following unbiased estimator:

Example. Consider a model generating solutions to a problem where, out of *n*=10 samples, *c*=3 pass all unit tests. With *k*=1, pass@1 = 0.30: the probability that a single randomly drawn sample is correct. With *k*=5, applying the unbiased estimator yields pass@5 = 0.92: by considering 5 candidates, the probability of at least one correct solution rises substantially. This distributional view provides a more robust and statistically grounded estimate of model capability than any single-output evaluation.

Software Engineering for Artificial Intelligence: LLM Evaluation

Constrain & Guardrails

So far we have discussed how to evaluate outputs after they are generated. A different and often more practical approach is to act earlier in the process: by constraining what the model can generate, we transform an ambiguous task into one where correctness can be checked automatically.

The underlying idea is engineering-driven. In the absence of a formal oracle, constraints serve as an artificial specification, that is, a set of rules that the output must satisfy, against which testing becomes straightforward and "deterministic".

In practice, this can be achieved at different levels.

Prompt-based

At the prompt level, explicit instructions are used to force the model toward parsable outputs, such as requiring only numeric responses or prohibiting abbreviations.

Few-shot prompting

Few-shot examples implicitly define the expected output schema without listing rules explicitly

Structured generation

At the generation level, libraries like Outlines use grammars and finite-state automata to enforce syntactic constraints directly during decoding, making non-compliant outputs structurally impossible.



Software Engineering for Artificial Intelligence

Model Validation and Testing in the Era of Foundation Models

